

লক্ষ্য

আপনি উপরের কোড ব্যবহার করে **MySQL + Node/Express API + React (Axios)** দিয়ে একটি সম্পূর্ণ CRUD (Create, Read, Update, Delete) অ্যাপ বানাবেন। ডাটাবেস চালাতে আমরা **XAMPP** এর MySQL সার্ভার (MariaDB) এবং **phpMyAdmin** ব্যবহার করবো।

আর্কিটেকচার ওভারভিউ

React (Vite/CRA) → Axios HTTP → Node/Express API → MySQL (XAMPP) → phpMyAdmin

- **Frontend (React):** লিস্ট দেখাবে, নতুন ডাটা যোগ করবে, Edit/Update/Delete করবে। - **Backend (Express):** REST API (GET/POST/PUT/DELETE) দিয়ে MySQL টেবিলের সাথে কাজ করবে। - **Database (MySQL):** XAMPP এর MySQL সার্ভার; phpMyAdmin দিয়ে টেবিল/ডাটা ম্যানেজ।

প্রয়োজনীয় সফটওয়্যার

1. **XAMPP** (Windows/Linux/Mac) – Apache + MySQL (MariaDB) + phpMyAdmin
2. **Node.js** (v18+ ভালো)
3. **কোড এডিটর** (VS Code)

ধাপ ১: XAMPP/MySQL সেটআপ

1. **XAMPP চালু করুন:** XAMPP Control Panel → **MySQL** (Start)! Apache লাগবে না, তবে চালু থাকলে সমস্যা নেই।
2. **phpMyAdmin ওপেন করুন:** ব্রাউজারে যান: `http://localhost/phpmyadmin`।
3. **ডাটাবেস বানান:**
4. বাম পাশে **New** → **Database name:** `mugdhotest` → **Create**।
5. **টেবিল বানান:** `mugdhotest` ডাটাবেসে গিয়ে **SQL** ট্যাবে নিচের স্ক্রিপ্ট রান করুন:

```
CREATE TABLE IF NOT EXISTS `murgi` (  
  `ID` INT NOT NULL AUTO_INCREMENT,  
  `Name` VARCHAR(100) NOT NULL,  
  `AGE` INT NOT NULL,  
  PRIMARY KEY (`ID`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

নোট: `root` ইউজার এবং **খালি পাসওয়ার্ড** দিয়ে XAMPP এ ডিফল্ট থাকে। প্রোডাকশনে পাসওয়ার্ড সেট করবেন।

(ঐচ্ছিক) Auto-Increment 3 থেকে শুরু করতে চান?

- শুধু পরের ইনসার্টগুলো 3 থেকে শুরু করতে:

```
ALTER TABLE `murgi` AUTO_INCREMENT = 3;
```

- সব ডাটা মুছে নতুন করে 3 থেকে শুরু:

```
TRUNCATE TABLE `murgi`;  
ALTER TABLE `murgi` AUTO_INCREMENT = 3;
```

ধাপ ২: Backend (Node/Express + MySQL) তৈরি

1. প্রজেক্ট ফোল্ডার তৈরি করুন (যেমন: `server/`) এবং টার্মিনালে যান।
2. নিচের কমান্ড চালান:

```
npm init -y  
npm i express mysql cors
```

3. `index.js` নামে একটি ফাইল বানিয়ে নিচের কোড রাখুন (আপনার দেয়া কোড সংগঠিত করা হল):

```
const express = require('express');  
const mysql = require('mysql');  
const cors = require('cors');  
  
const app = express();  
app.use(cors());  
app.use(express.json()); // JSON body পার্স করার জন্য  
  
const db = mysql.createConnection({  
  host: 'localhost',  
  user: 'root',  
  password: '',  
  database: 'mugdhotest'  
});  
  
// READ: সব নাম  
app.get('/names', (req, res) => {  
  const sql = 'SELECT * FROM `murgi`';  
  db.query(sql, (err, data) => {  
    if (err) return res.status(500).json(err);  
    return res.json(data);  
  });  
});  
  
// CREATE: নতুন নাম যোগ
```

```

app.post('/names', (req, res) => {
  const { Name, AGE } = req.body;
  const sql = 'INSERT INTO `murgi` (`Name`, `AGE`) VALUES (?, ?)';
  db.query(sql, [Name, AGE], (err, result) => {
    if (err) return res.status(500).json(err);
    const newItem = { ID: result.insertId, Name, AGE };
    return res.json(newItem);
  });
});

// DELETE: আইডি দিয়ে ডিলিট
app.delete('/names/:id', (req, res) => {
  const { id } = req.params;
  const sql = 'DELETE FROM `murgi` WHERE ID = ?';
  db.query(sql, [id], (err) => {
    if (err) return res.status(500).json(err);
    return res.json({ message: 'Deleted successfully' });
  });
});

// UPDATE: আইডি দিয়ে আপডেট
app.put('/names/:id', (req, res) => {
  const { id } = req.params;
  const { Name, AGE } = req.body;

  // \\ বিকল্প ভ্যালিডেশন (|| ছাড়া):
  const filled = [Name, AGE].every(v => String(v ?? '').trim() !== '');
  if (!filled) return res.status(400).json({ error: 'Name এবং AGE বাধ্যতামূলক' });

  const sql = 'UPDATE `murgi` SET `Name` = ?, `AGE` = ? WHERE `ID` = ?';
  db.query(sql, [Name, AGE, id], (err) => {
    if (err) return res.status(500).json(err);
    return res.json({ ID: Number(id), Name, AGE });
  });
});

app.get('/', (req, res) => {
  res.json('From Backend Side');
});

app.listen(4000, () => {
  console.log('listening on http://localhost:4000');
});

```

1. সার্ভার চালু করুন:

```

node index.js
# বা nodemon থাকলে: npx nodemon index.js

```

2. টার্মিনালে `listening on http://localhost:4000` দেখাবে।

API টেস্ট (ঐচ্ছিক)

- **GET:** `http://localhost:4000/names`
- **POST:** `http://localhost:4000/names` (JSON body: `{ "Name": "kashem", "AGE": 26 }`)
- **PUT:** `http://localhost:4000/names/3`
- **DELETE:** `http://localhost:4000/names/3`

ধাপ ৩: Frontend (React + Axios) তৈরি

আপনি Vite বা CRA যেকোনোটা নিতে পারেন। নিচে Vite দেখানো হলো।

1. প্রজেক্ট তৈরি:

```
# এক ধাপ পিছনে গিয়ে নতুন ফোল্ডার করুন
npm create vite@latest client -- --template react
cd client
npm i
npm i axios
```

2. `src/App.jsx` (অথবা আলাদা `Home.jsx`) এ নিচের কম্পোনেন্ট দিন। আপনার দেয়া কোডকে একটু গুছিয়ে দেয়া হলো:

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';

export default function App() {
  const [data, setData] = useState([]);
  const [name, setName] = useState('');
  const [age, setAge] = useState('');
  const [editId, setEditId] = useState(null);

  useEffect(() => {
    axios.get('http://localhost:4000/names')
      .then(res => setData(res.data))
      .catch(console.error);
  }, []);

  const handleAdd = () => {
    axios.post('http://localhost:4000/names', { Name: name, AGE: age })
      .then(res => {
        setData([...data, res.data]);
        setName('');
        setAge('');
      })
      .catch(console.error);
  }
}
```

```

};

const handleDelete = (id) => {
  axios.delete(`http://localhost:4000/names/${id}`)
    .then(() => setData(data.filter(item => item.ID !== id)))
    .catch(console.error);
};

return (
  <div className="p-6 max-w-xl mx-auto">
    <h1 className="text-2xl font-bold mb-4">Names</h1>

    <ul className="space-y-2 mb-6">
      {data.map(item => (
        <li key={item.ID} className="border p-3 rounded">
          <span>{item.Name}  Age: {item.AGE}</span>

          <button
            onClick={() => { setEditId(item.ID); setName(item.Name);
setAge(item.AGE); }}
            className="ml-2 bg-yellow-500 text-white px-2 py-1 rounded"
            >Edit</button>

          {editId === item.ID && (
            <button
              onClick={() => {
                axios.put(`http://localhost:4000/names/${editId}`, { Name:
name, AGE: age })
                  .then(res => {
                    setData(data.map(d => (d.ID === editId ? res.data :
d)));

                    setEditId(null);
                    setName('');
                    setAge('');
                  })
                  .catch(console.error);
                }}
              className="ml-2 bg-green-600 text-white px-2 py-1 rounded"
              >Update</button>
            )}

            <button
              onClick={() => handleDelete(item.ID)}
              className="ml-2 bg-red-600 text-white px-2 py-1 rounded"
              >Delete</button>
          </li>
        ))}
      </ul>

      <div className="mb-4 space-x-2">
        <input

```

```

        type="text"
        placeholder="Name"
        value={name}
        onChange={e => setName(e.target.value)}
        className="border p-1 rounded"
      />
      <input
        type="number"
        placeholder="Age"
        value={age}
        onChange={e => setAge(e.target.value)}
        className="border p-1 rounded"
      />
      <button onClick={handleAdd} className="bg-blue-600 text-white px-3
py-1 rounded">Add</button>
    </div>
  </div>
);
}

```

1. ফ্রন্টএন্ড চালু করুন:

```
npm run dev
```

2. সাধারণত Vite <http://localhost:5173> তে চলে।

CORS টিপ: ব্যাকএন্ডে `app.use(cors())` আছে। দরকার হলে নির্দিষ্ট অরিজিন দিন:

```
app.use(cors({ origin: 'http://localhost:5173' }));
```

ধাপ ৪: Edit/Update বোতামের লজিক (এক আইটেমে দেখাবে)

আপনি ঠিকই করেছেন—যে আইটেম **Edit** করছেন, কেবল সেই আইটেমের জন্যই **Update** বোতাম দেখা যাবে:

```

{editId === item.ID && (
  <button onClick={...}>Update</button>
)}

```

ধাপ ৫: ভ্যালিডেশন (|| ছাড়া বিকল্প)

ব্যাকএন্ডে `if (!Name || !AGE)` এর বদলে নিচেরটা ব্যবহার করলে **logical OR (||)** সরাসরি ব্যবহার করতে হবে না:

```
const filled = [Name, AGE].every(v => String(v ?? '').trim() !== '');
if (!filled) return res.status(400).json({ error: 'Name এবং AGE বাধ্যতামূলক' });
```

একইভাবে ফ্রন্টএন্ডেও সাবমিটের আগে চেক করতে পারেন।

ধাপ ৬: কমন এরর ও সমাধান

1. `AxiosError: Network Error` → ব্যাকএন্ড চলছে তো? (`node index.js`)। CORS লাগছে কি? পোর্ট ঠিক আছে কি?
2. `ER_ACCESS_DENIED_ERROR` → MySQL ইউজার/পাস ঠিক রাখুন। XAMPP এ সাধারণত `root` /খালি পাসওয়ার্ড।
3. `Cannot POST /names` → `app.use(express.json())` আছে তো? রুট পথ সঠিক তো?
4. **ID 6,7,8 থেকে শুরু হচ্ছে** → উপরে দেখানো `ALTER TABLE ... AUTO_INCREMENT = 3;` চালান। পুরনো ডাটা থাকলে `TRUNCATE` করে নিন।
5. **পোর্ট কনফ্লিক্ট** → 4000 ব্যস্ত থাকলে 5000 নিন; React 5173 ব্যস্ত হলে 5174 নিন।

ধাপ ৭: সিকিউরিটি/প্রোডাকশন টিপস

- `.env` ফাইল করে DB ক্রেডেনশিয়াল রাখুন (লোকাল ডেমোতে না হলেও হবে):

```
npm i dotenv
```

```
require('dotenv').config();
const db = mysql.createConnection({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
});
```

- Prepared statements (আমরা ? প্লেসহোল্ডার ব্যবহার করেছি)।
- ইনপুট স্যানিটাইজ/ভ্যালিডেট করুন।

কুইক চেকলিস্ট

- [] XAMPP → MySQL **Start**
- [] phpMyAdmin → DB `mugdhotest` + Table `murgi`
- [] Backend `node index.js` → `http://localhost:4000`
- [] Frontend `npm run dev` → `http://localhost:5173`
- [] Create/Read/Update/Delete কাজ করছে

সম্পূর্ণ সোর্স (রেফারেন্স)

এই সেকশনে উপরে দেওয়া **Backend** `index.js` এবং **Frontend** `App.jsx`-এর গুছানো ভার্সন রয়েছে, কপি-পেস্টে চলবে।

Backend: `index.js`

(উপরের ধাপ ২-এর কোডই ব্যবহার করুন)

Frontend: `src/App.jsx`

(উপরের ধাপ ৩-এর কোডই ব্যবহার করুন)

শেষ কথা

এভাবেই XAMPP + phpMyAdmin, Node/Express API, আর React (Axios) দিয়ে ফুল CRUD অ্যাপ তৈরি হয়। আপনার দেয়া কোডে ছোটখাট সংগঠন/ভ্যালিডেশন যোগ করা হয়েছে। কোনো জায়গায় আটকে গেলে ওপরের **কমন এরর** সেকশন দেখে নিন।